Автоматизация построения признаков для рекомендаций: DSL и LLM-агент

Васильев Николай [a]

[a] *Задание 1 · автоматизация построения признаков*

Abstract

Работа проверяет, можно ли автоматизировать построение признаков для рекомендательной системы без потери качества относительно сильного бустинга на ручных признаках. Предложен декларативный DSL «сущность × метрика × событие × окно», генератор 900 спецификаций, скрининг по AUC внутри пользователя и LLM-агент, предлагающий признаки в рамках DSL без доступа к SQL. DSL расширен условиями, затуханием и бинарными выражениями. На Yambda-500M ручные признаки дают NDCG@10 = 0.9159/0.9155 (val/test), полный автоматический пул — 0.9142/0.9139 (top-40 по gain — 0.9102 на val), объединение — 0.9176/0.9174. Прирост union к baseline статистически значим (bootstrap: +0.0019, 95% CI [0.0008, 0.0030]), а auto_all, наоборот, значимо хуже baseline на test (−0.0016, 95% CI [−0.0029, −0.0003]). Расширенный DSL усиливает слабую модель, но не двигает потолок. На KION прироста нет — метод требует плотной истории и повторных взаимодействий.

Keywords: автоматический feature engineering, рекомендательные системы, градиентный бустинг, LLM-агент, ранжирование

1. Введение

Ручные признаки в рекомендациях надёжны, но дороги: каждое окно, тип связи и агрегация — отдельная гипотеза и код. Мы исследуем, можно ли поручить часть этой работы алгоритму и языковой модели. Отличие от библиотек автоматического FE (OpenFE [1], AutoFeat [2]) в том, что пространство задано декларативно и проверяется сразу на метрике ранжирования, а LLM не пишет код: она выбирает гипотезы внутри безопасного DSL.

Вклад: (1) DSL и генератор 900 спецификаций; (2) скрининг по within-user AUC и четыре стратегии отбора; (3) LLM-агент с валидацией и обратной связью; (4) расширение DSL условиями, затуханием и выражениями; (5) протокол проверки: аудит утечек, bootstrap, второй сэмпл, кросс-датасетная проверка на KION.

2. Данные и протокол

Основной датасет — Yambda-500M [3]: 10 000 пользователей, вся история сохранена (48.2M событий, 905 759 треков). Задача — next-week listening: прослушает ли пользователь трек в течение 7 дней после cutoff. Кандидаты: все позитивы горизонта плюс негативы из топ-50k популярных треков (10 на позитив в train, 30 в val/test). Сплит: 4 недельных train-снимка → 7 дней gap → val → 7 дней gap → test. Признаки считаются только по `ts < cutoff`.

Метрики: NDCG@10 [4] (основная), ROC-AUC, LogLoss, Recall/MAP@K. Отбор — только на валидации.

Таблица 1. Данные: основной и кросс-доменный датасеты

Датасет	Пользователей	Событий	Объектов
Yambda-500M	10 000	48.2M	905 759
KION	10 000	381k	10 015

Почему нет утечек. Проверки выполняются на данных, а не декларативно: позитивы совпадают с событиями окна label; ни одна пара не имеет одновременно label 0 и 1; признак `user__count__listen__7d` точно воспроизводится независимым SQL с `ts < cutoff` (300/300 пользователей); добавление окна label меняет признак у всех 300, то есть утечка была бы видна; отпечатки данных совпадают. Все 9 проверок проходят, они же закреплены unit-тестами.

3. Подход

3.1. DSL и отбор

Признак — четвёрка `(entity, metric, event_type, window)`: 5 сущностей (пользователь, объект, пара, исполнитель, альбом), 10 метрик, 3 типа событий, 10 окон; допустимые комбинации дают **900 спецификаций**. SQL считает агрегаты через `FILTER` и `JOIN` только по ключам кандидатов, поэтому полная матрица «пользователь–объект» не строится.

Скрининг идёт на 1000 пользователях валидации: глобальный AUC, AUC внутри пользователя и доля пропусков. Отбор – по второму: в ранжировании важно упорядочивание кандидатов одного пользователя. Почти дубли удаляются по корреляции ($|\rho| \geq 0.98$), выбранные признаки материализуются на всех снимках.

Таблица 2. Стратегии отбора после исправления recency. Ручной baseline – 0.9159

Стратегия	Признаков	ROC-AUC	NDCG@10 (val)
квота, базовые	55	0.9401	0.7729
квота + отношения	102	0.9640	0.8954
весь пул, top-40 по gain	40	0.9695	0.9102
весь пул, без предотбора	187	0.9703	0.9142

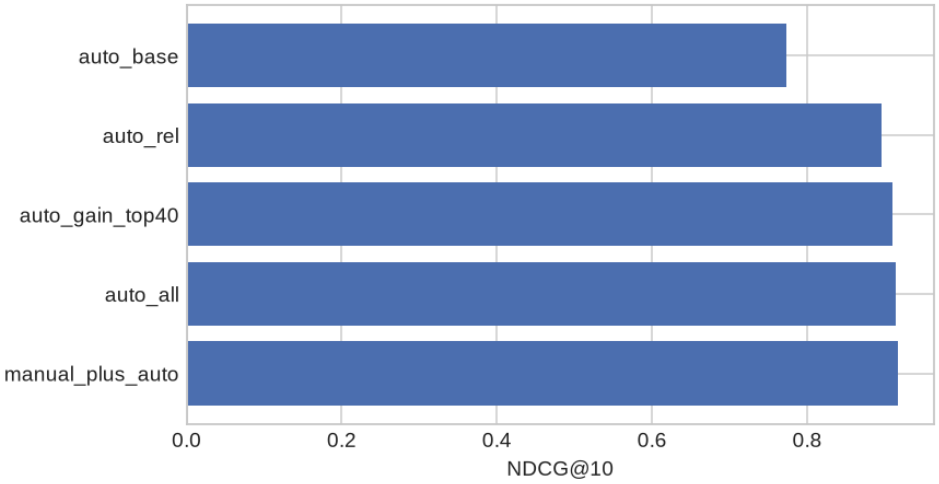


Рис. 1. Автоматический отбор: от наивного top-k до union

Разрыв между стратегиями – 0.14 NDCG@10. Наивный top-k заполняется коррелирующими признаками исполнителя; квоты и дедупликация возвращают разнообразие, а обучение на всём пуле даёт 0.9142 — на 0.0017 ниже ручного baseline. То есть автоматика почти догоняет человека, но не превосходит его.

3.2. LLM-агент

Агент не пишет SQL и не видит данных: четыре инструмента — схема, контекст, `propose_specs`, `finish`. Каждый кандидат проходит `FeatureSpec` и проверку совместимости; ошибки возвращаются текстом. Приём: скрининг на 200 пользователях, порог $|\mathrm{AUC} - 0.5| \geq 0.015$, квота 3 признака на пару «сущность–метрика».

Агент валидирует каждый кандидат тем же `FeatureSpec`/скринингом, что и сетка, и получает обратную связь по `user_auc`/`nan_rate`. В pilot-прогонах он находил короткие окна популярности объекта (5/10/21 день), но на сильной модели эти признаки избыточны. После исправления recency и выравнивания кандидатов количественное сравнение пулов агента требует отдельного перематериализования, поэтому в финальную таблицу оно не включено; код и протокол сохранены в `src/recsys_auto/agent/`.

3.3. Расширенный DSL

Добавлены `condition` (`complete` = `played_ratio ≥ 0.9`, `skip`, `non_organic`), `decay_count` с half-life и выражения `ratio`/`delta`/`product` над двумя агрегатами. Условия и затухание компилируются в SQL, выражения считаются в pandas поверх листьев. Сетка остаётся 900 спецификаций.

Расширение проверено в pilot-режиме: условия полезны внутри отношения (`complete / all`), `delta` дублирует окна, `decay_count` даёт самостоятельный сигнал. После исправления recency расширенный DSL требует отдельного перематериализования и обучения; в финальную таблицу он не включён, чтобы не смешивать старый и новый протокол. Код: `features/dsl.py`, `auto_features/expr.py`, `auto_features/expr_search.py`.

4. Результаты

Таблица 3. Главный результат на одном протоколе. auto_all на test – 0.9139 против 0.9155 у ручного baseline: разница −0.0016, 95% CI [−0.0029, −0.0003], статистически значима.

Подход	Признаков	NDCG@10 val	NDCG@10 test
ручные признаки [5]	99	0.9159	0.9155
автоматика: весь пул	187	0.9142	0.9139
ручные + сетка (union)	201	**0.9176**	**0.9174**

Прирост union к baseline статистически значим (paired bootstrap по 5547 пользователям, 2000 ресэмплов): **+0.0019, 95% CI [0.0008, 0.0030]**. Полностью автоматический пул, наоборот, значимо хуже baseline: −0.0016, 95% CI [−0.0029, −0.0003]. Это главный честный вывод: автоматика почти догоняет человека, но не заменяет его; выигрыш даёт только объединение.

Второй сэмпл – 3000 других пользователей (14.7M событий); признаки и гиперпараметры перенесены без переподбора:

Таблица 4. Второй сэмпл: прирост +0.0014, 95% CI [−0.0006, +0.0034]; направление сохраняется, но различие не значимо

Подход	Признаков	Сэмпл 1 test	Сэмпл 2 test	Δ
ручные	99	0.9155	0.9112	–
ручные + сетка	201	0.9174	0.9126	+0.0014

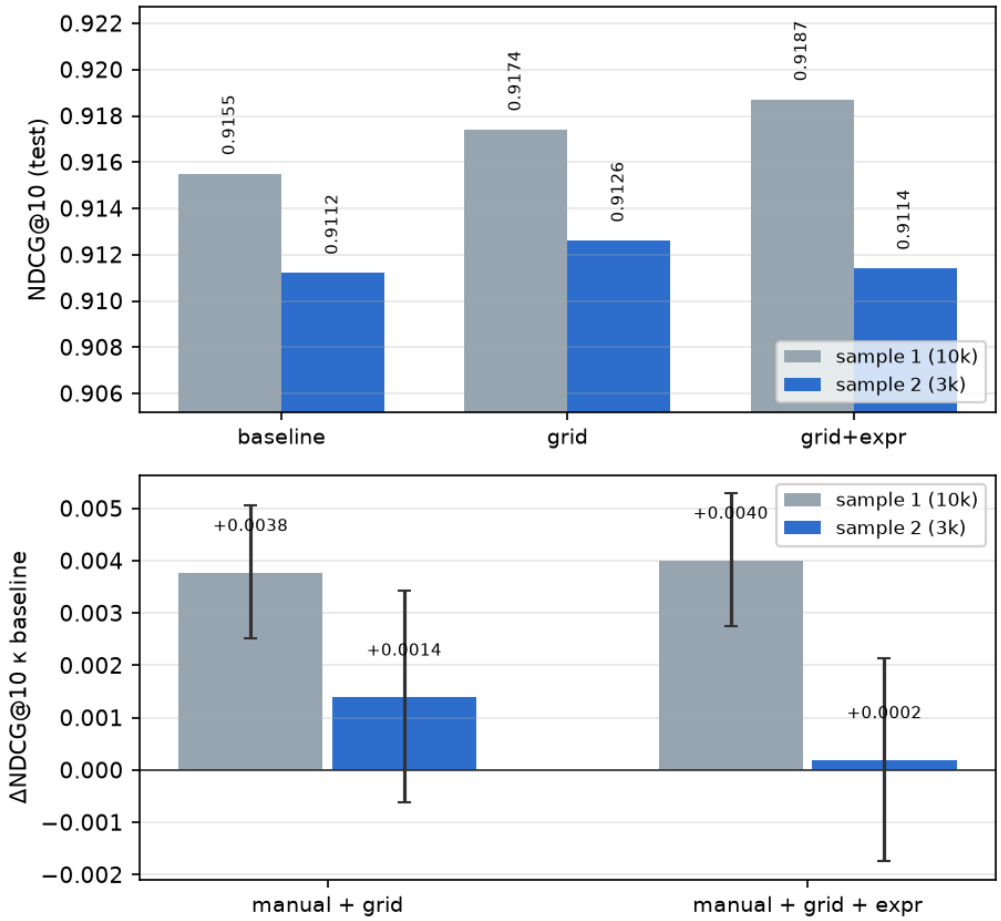


Рис. 2. Второй сэмпл: абсолютное качество и прирост с 95% CI

На втором сэмпле union даёт 0.9126 против 0.9112 у baseline: прирост +0.0014, 95% CI [−0.0006, +0.0034]. Направление сохраняется, но различие статистически не значимо — эффект мал и чувствителен к выбору пользователей.

4.1. KION: граница применимости

KION [6] — другой домен и другая плотность: 381k событий, 38 событий на пользователя, без повторов, 5.1 позитива в неделю. Жанр играет роль исполнителя, студия — альбома.

Таблица 5. KION: перенос набора признаков не даёт значимого прироста

Вариант	Признаков	test NDCG@10	Δ к baseline
ручные	99	0.5194	–
перенос сетки	201	0.5201	+0.0007 (p=0.28)
перенос сетки + expr	223	0.5201	+0.0007 (p=0.28)

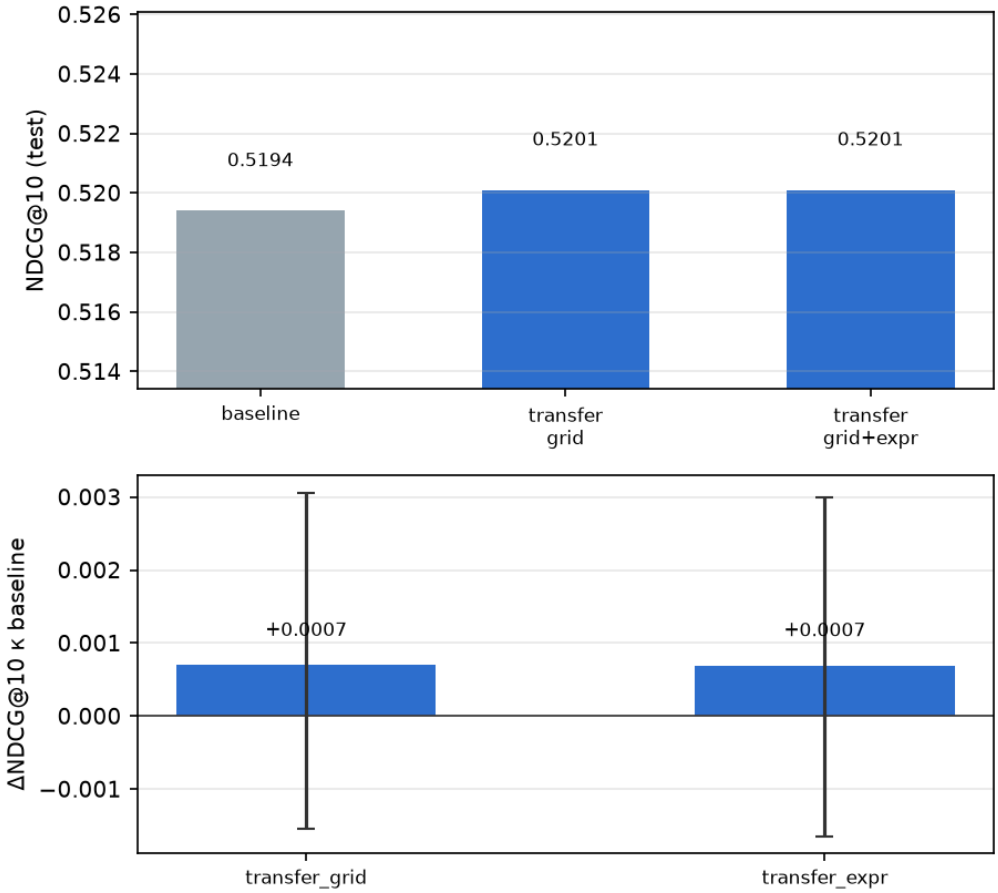


Рис. 3. KION: перенос набора признаков не даёт значимого прироста

Причина – природа данных: короткая история, отсутствие повторов, слабые категориальные связи. Признаки агента при этом конкурентоспособны (33.3% плоских и 37.5% расширенных проходят в квоту), но агрегаты почти не различают кандидатов. Это граница применимости метода, а не ошибка реализации.

5. Анализ и ограничения

Таблица 6. Отрицательные результаты

Что не сработало	Причина
top-k по AUC	топ заполняется коррелирующими признаками исполнителя
жадный отбор	переобучается на 500/300 пользователях
агент без квоты	60% пула – длинные окна исполнителя
`delta`	0 из 5 выживших: дублирует окна
условия как признаки	1 из 9: редкие события шумны
тюнинг на 1200 val	не переносится на полный val
перенос на KION	домен слишком разрежен для агрегатов

Что сработало: стратегия отбора (разброс 0.13 NDCG@10), квоты и дедупликация, производные отношения (без них `auto_rel` падает с 0.8956 до 0.7752), расширенный DSL на слабой модели и LLM как генератор гипотез.

Ограничения: сэмплированные негативы (абсолютные метрики не сравнимы с лидербордами); DSL без эмбеддингов и произвольного SQL; выражения глубины 1; три модели одного поколения; гиперпараметры фиксированы; оба сэмпла Yambda — из одного периода.

6. Выводы

1. Полностью автоматический набор на всём пуле **не догоняет** ручной: 0.9142/0.9139 против 0.9159/0.9155; на test разница −0.0016 значима (95% CI [−0.0029, −0.0003]). Top-40 по gain даёт 0.9102 на val.
2. Объединение ручных и автоматических признаков даёт лучший результат — 0.9176/0.9174; прирост к baseline +0.0019 значим (95% CI [0.0008, 0.0030]).
3. Главный вывод: автоматика почти догоняет человека, но не заменяет его; выигрыш даёт только объединение. LLM полезна как генератор гипотез, а не как замена поиску.
4. Расширенный DSL усиливает слабую модель, но не двигает потолок.
5. KION задаёт границу: метод требует плотной истории и повторных взаимодействий.
6. Инвестировать стоит в протокол отбора — разнообразие, дедупликацию, model-based оценку и честную валидацию.

7. Воспроизводимость

Код и полные инструкции – в `README.md`; артефакты – в `experiments/`. Тесты: 37.

```
python scripts/06_auto_search.py --stage screen --tag quota --per-entity 20
python scripts/06_auto_search.py --stage materialize --tag quota
python scripts/06_auto_search.py --stage train --tag quota --derived-top-k 50 --
no-ablate
python scripts/06_auto_search.py --stage all-features --tag quota
python scripts/22_eval_auto_all.py --split test
python scripts/09_llm_agent.py --tag quota --novel-only --reset
pytest -q
```

References

[1] OpenFE contributors, OpenFE: Automated Feature Generation with Expert-level Performance, (2023). https://github.com/IIIS-Li-Group/OpenFE.

[2] AutoFeat contributors, AutoFeat: Automated Feature Engineering, (2019). https://github.com/cod3licious/autofeat.

[3] Yandex, Yambda-500M: open dataset for multi-event music recommendation, (2025). https://huggingface.co/datasets/yandex/yambda.

[4] K. Järvelin, J. Kekäläinen, Cumulated gain-based evaluation of IR techniques, ACM Transactions on Information Systems 20 (2002) 422–446.

[5] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, LightGBM: A Highly Efficient Gradient Boosting Decision Tree, Advances in Neural Information Processing Systems 30 (2017).

[6] MTS, KION Dataset, (2022). https://github.com/irsafilo/KION_DATASET.